

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 10 luglio 2012 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

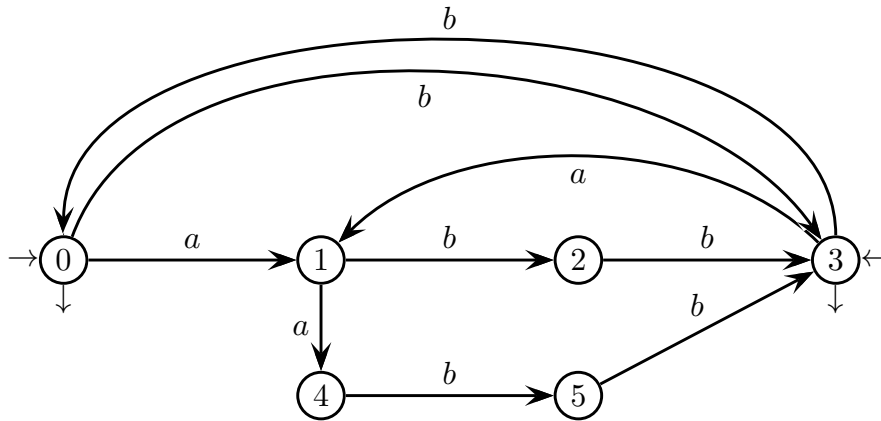
FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

Si consideri il seguente automa nondeterministico (avente due stati iniziali):

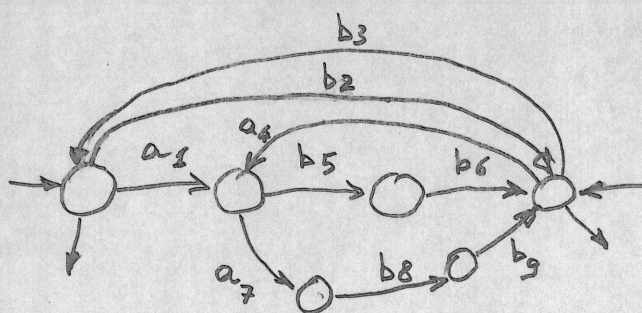


Si risponda alle domande seguenti:

1. Utilizzando il procedimento di determinizzazione basato sull'algoritmo Berry-Sethi, se ne produca una versione equivalente deterministica.
2. Si verifichi se l'automa deterministico ottenuto al punto precedente è minimo; in caso negativo si ricavi, mediante un procedimento sistematico, l'automa minimo equivalente.
3. Applicando il metodo BMC all'automa minimo, si ricavi l'espressione regolare equivalente a tale automa.
4. Si consideri il seguente linguaggio L di alfabeto $\{a, b\}$, composto da tutte e sole le stringhe che soddisfano la condizione: ogni carattere a presente nella stringa è seguito da almeno due caratteri e il secondo di questi è b . Per esempio, fanno parte del linguaggio le stringhe $\varepsilon, bb, babbb, baabb$, mentre non ne fanno parte le stringhe $bab, aaa, abab$. Scrivere l'espressione regolare che definisce tale linguaggio, giustificando adeguatamente la risposta

Soluzione

Le soluzioni sono tratteggiate nelle seguenti figure.



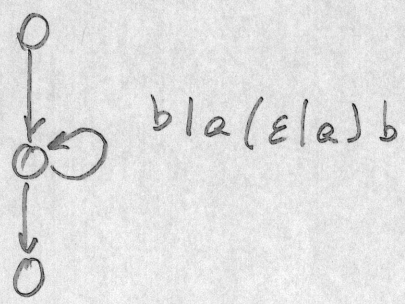
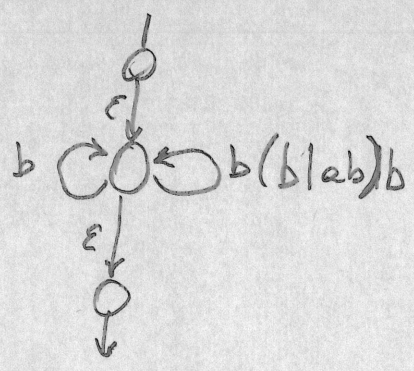
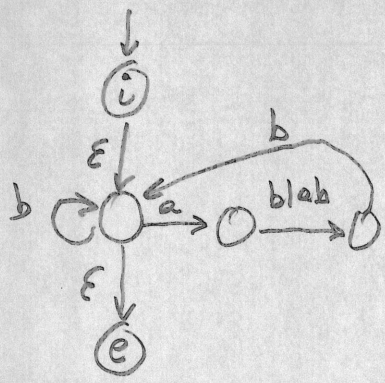
$ini(L) =$

$$ini(L) = \{a_1 a_4 b_2 b_3 \rightarrow\}$$

Simb	Seguiti
a_1	$b_5 a_7$
b_2	$b_3 a_4 \rightarrow$
b_3	$a_1 b_2 \rightarrow$
a_4	$b_5 a_7$
b_5	b_6
b_6	$b_3 a_4 \rightarrow$
a_7	b_8
b_8	b_9
b_9	$b_3 a_4$

Si noti infine che il linguaggio L specificato al punto non è altro che il linguaggio riconosciuto dall'automa dato.

3



$$R = (b|a|abb|aabb)^*$$

2 Grammatiche libere e automi a pila 20%

1. Si consideri il linguaggio L così specificato:

- 1- una frase è una lista di uno o più *costrutti*
- 2- i costrutti sono separati e terminati da $\#$
- 3- ogni costrutto è del tipo a^*b^*
- 4- almeno un costrutto ha la forma $a^n b^n$, $n \geq 0$
- 5- almeno un costrutto è diverso da ε .

Es.: $ab^2\#ab\# \#b\#$

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G , non *EBNF* per il linguaggio L e si disegni l'albero sintattici dell'esempio dato sopra.
 - (b) Si verifichi se G è ambigua.
 - (c) (facoltativa) Se G fosse ambigua, si scriva una grammatica equivalente non ambigua.
-

Soluzione

Nello scrivere la grammatica conviene aderire punto per punto alle specifiche (1 - 5). Come prima approssimazione si scrive la grammatica (chiaramente non ambigua) che considera soltanto le specifiche 1,2,3:

$$\begin{aligned} S &\rightarrow C\#S \mid C\# \\ C &\rightarrow AB \mid \varepsilon \\ A &\rightarrow aA \mid a \\ B &\rightarrow bB \mid b \end{aligned}$$

I punti 4 e 5 impongono che nella lista esista un costrutto con la proprietà 4 e uno con la proprietà 5. Classifichiamo i costrutti nei vari casi rilevanti, denotati dai nonterminali $C_0, \dots, C_4$:

$$\begin{aligned} C_0 &\rightarrow \varepsilon \\ C_1 &\rightarrow aC_1 \mid a \\ C_2 &\rightarrow bC_2 \mid b \\ C_3 &\rightarrow aC_3b \mid ab \\ C_4 &\rightarrow C_1C_3 \mid C_3C_2 \end{aligned}$$

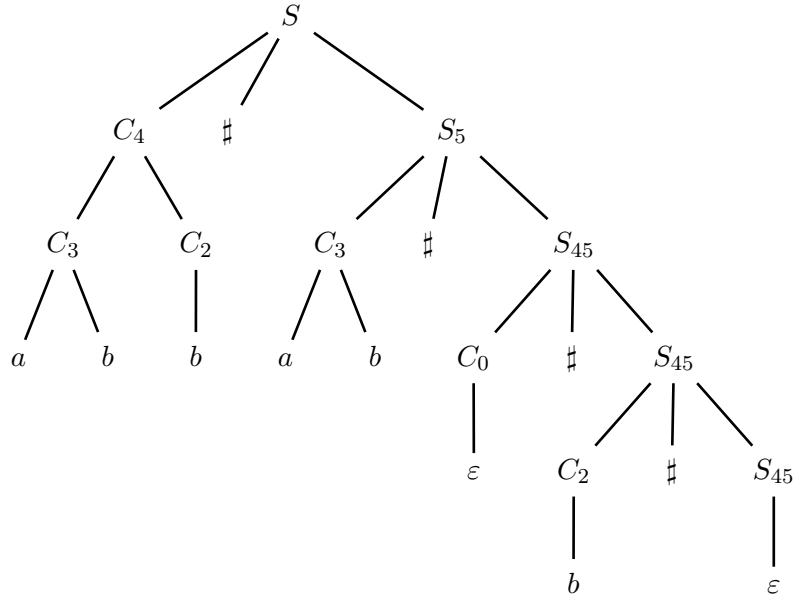
Ora nella costruzione della lista dobbiamo distinguere più situazioni, introducendo dei corrispettivi nonterminali, come sotto indicato.

- S_4 : la lista già generata soddisfa il punto 4-
- S_5 : la lista già generata soddisfa il punto 5-
- S_{45} : la lista già generata soddisfa i punti 4- e 5-

Le regole corrispondenti sono:

$$\begin{aligned} S &\rightarrow C_0\#S_4 \mid C_3\#S_{45} \mid C_1\#S_5 \mid C_2\#S_5 \mid C_4\#S_5 \\ S_4 &\rightarrow C_0\#S_4 \mid C_1\#S_{45} \mid C_2\#S_{45} \mid C_3\#S_{45} \mid C_4\#S_{45} \\ S_5 &\rightarrow C_0\#S_{45} \mid C_3\#S_{45} \\ S_{45} &\rightarrow C_0\#S_{45} \mid C_1\#S_{45} \mid C_2\#S_{45} \mid C_3\#S_{45} \mid C_4\#S_{45} \mid \varepsilon \end{aligned}$$

L'albero sintattico di $ab^2\#ab\# \#b\#$ è



La grammatica così costruita non è ambigua. Infatti ogni costrutto compreso tra due diesis ammette un solo modo per essere ridotto a un nonterminale tra $C_0, \dots, C_4$. Inoltre, analizzando da sin. a destra la stringa (forma di frase)

$$C_{i_1} \# C_{i_2} \# \dots \# C_{i_n} \#$$

si vede che a ogni passo la scelta della derivazione sinistra è unica, essendo imposta dal nonterminale C_{i_k} sotto esame.

2. Un linguaggio di programmazione per applicazioni gestionali possiede i seguenti tipi elementari: carattere (indicato con l'identificatore predefinito *c*), intero (*i*), floating (*f*), data (*d*), tempo (*t*), esadecimale (*x*). Le variabili vengono dichiarate con una frase che inizia con la parola chiave **data** (nel seguito le parole chiave sono sempre indicate in grassetto), mentre la parola chiave **type** introduce il tipo della variabile in questione:

```
data x type i;      -- variabile intera x
```

Le variabili possono essere anche array dei soli tipi elementari:

```
data customer_name(25) type c;  -- variabile customer_name array di 25 car.
```

E' possibile definire nuovi tipi mediante frasi che iniziano con la parola chiave **types**. I nomi di tali tipi possono essere usati per dichiarare variabili, come negli usuali linguaggi di programmazione tipizzati:

```
types age type i;
types t_name(25) type c;
data customer_name type t_name;
data customer_age type age;
```

Tra i tipi non elementari ci sono i record. Come gli array, anche i record possono essere dichiarati sia mediante un tipo record (utilizzabile nelle dichiarazioni di variabili), sia direttamente mediante una variabile di tipo record. Si noti che il tipo di un campo può essere omesso, nel qual caso si intende sia *c*. I campi dei record possono essere di tipo qualsiasi.

<pre>data begin of <i>booking</i> -- dich. della -- var. booking di tipo record a 3 campi <i>id</i>(4) type <i>c</i>; <i>flight_date</i> type <i>d</i>; <i>name</i> type <i>customer_name</i>; end of <i>booking</i>;</pre>	<pre>types begin of <i>personal_data</i> -- dich. -- del tipo personal_data record a 3 campi <i>name</i>(25); -- name campo di 25 car. <i>birth</i> type <i>d</i>; <i>city</i>(25); end of <i>personal_data</i>;</pre>
--	--

Un altro tipo non elementare è la tabella, che prevede un numero massimo di elementi dello stesso tipo; tale tipo può essere qualsiasi, elementare o non. Le dichiarazioni di tabelle (sia quelle di variabile, sia di tipo) si caratterizzano per la presenza della parola chiave **occurs** seguita dal numero di elementi previsti:

```
types colleagues type personal_data occurs 10; -- tipo tabella con 10 elem.
```

La sezione dichiarativa di un programma inizia con le dichiarazioni di tipi e variabili elementari e array. Seguono le definizioni di tipi e variabili record e tabella, combinate e alternate in qualsiasi modo. Per esempio la seguente dichiarazione definisce un record *employee* con un campo *phone* che è una tabella (definita sul posto) di 5 record di tipo *phone_fax_number* (tipo record definito a parte), e un campo *office_mates* di tipo *colleagues*, tabella definita a parte:

```

types begin of phone_fax_number
country_code(3) type i;
area_code(3) type i;
number(10) type i;
end of phone_fax_number;

```

```

types begin of employee
name(25);
phone type phone_fax_number occurs 5;
office_mates type colleagues;
end of employee;

```

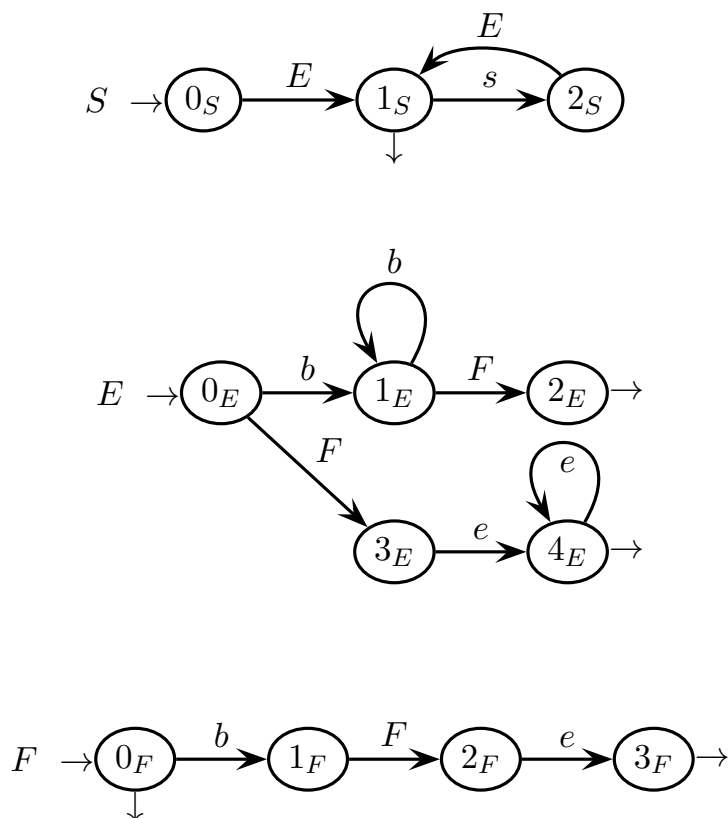
Si scriva una grammatica estesa (EBNF) non ambigua per questo linguaggio. Si suggerisce di usare il nonterminale $\langle DECL_SECT \rangle$ come assioma e di rappresentare tutti gli identificatori con il terminale id e tutti i numeri interi positivi col terminale $posInt$. Sono particolarmente apprezzate grammatiche semplici e ben strutturate.

Soluzione

$$\begin{aligned}
\langle EL_TYPE \rangle &\rightarrow 'c' \mid 'i' \mid 'f' \mid 't' \mid 'x' \\
\langle EL_ARR_DEF \rangle &\rightarrow id \text{ ['('posInt')']} type (\langle EL_TYPE \rangle \mid id); \\
\langle EL_ARR_DECL \rangle &\rightarrow (data \mid types) \langle EL_ARR_DEF \rangle \\
\langle TABLE_DEF \rangle &\rightarrow id type id occurs posInt; \\
\langle TABLE_DECL \rangle &\rightarrow (data \mid types) \langle TABLE_DEF \rangle \\
\langle RECORD_DEF \rangle &\rightarrow begin of id ((DEF) \mid id \text{ ['('posint')']})^+ end of id; \\
\langle RECORD_DECL \rangle &\rightarrow (data \mid types) \langle RECORD_DEF \rangle \\
\langle DEF \rangle &\rightarrow \langle EL_ARR_DEF \rangle \mid \langle TABLE_DEF \rangle \mid \langle RECORD_DEF \rangle \\
\langle DECL_SECT \rangle &\rightarrow \langle EL_ARR_DECL \rangle^* (\langle RECORD_DECL \rangle \mid \langle TABLE_DECL \rangle)^*
\end{aligned}$$

3 Analisi sintattica e parsificatori 20%

1. È data la rete di macchine ricorsive seguente (assioma S):



Si risponda alle domande seguenti:

- (a) Si costruisca l'automa pilota e si verifichi se la condizione $ELR(1)$ è soddisfatta in ogni macro-stato.

Per gli allievi che hanno frequentato in anni accademici precedenti.

Potete trasformare la rete nella grammatica

$$S \rightarrow EsS \mid E, \quad E \rightarrow BF \mid FH, \quad B \rightarrow bB \mid b, \quad H \rightarrow eH \mid e$$

e applicare il metodo di costruzione $LR(1)$.

- (b) Si verifichi se la rete soddisfa la condizione $ELL(1)$.

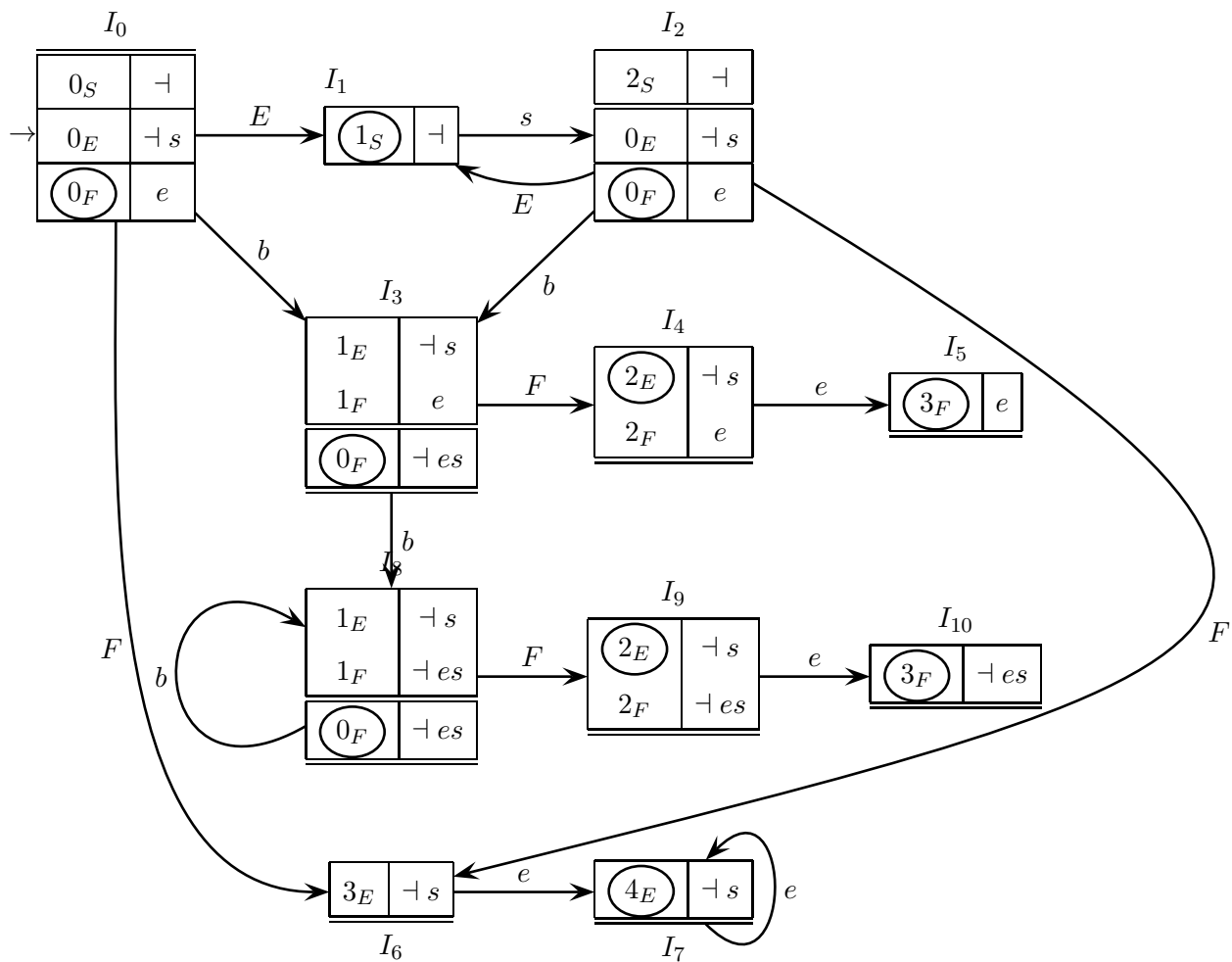
Per gli allievi che hanno frequentato in anni accademici precedenti.

Potete applicare il metodo tradizionale di calcolo degli insiemi guida.

- (c) (Facoltativo) Se la grammatica non fosse $ELL(1)$, si esamini se un valore $k > 1$ permette di togliere il non-determinismo.

Soluzione

Ecco il grafo pilota esteso della grammatica G :



Si vede subito che nei m-stati non ci sono conflitti riduzione-riduzione. Per i conflitti riduzione-spostamento, nei m-stati contenenti uno stato finale, la sua prospezione è disgiunta dai caratteri che possono essere letti in quel m-stato: ad es. in I_3 la riduzione $\varepsilon \rightsquigarrow F$ è prescritta dall'essere e , oppure s oppure $\neg$, il prossimo carattere, mentre lo spostamento legge b .

Soluzione con il metodo tradizionale. Costruendo l'automa pilota $LR(1)$ per la grammatica BNF data, si ottiene il seguente macro-stato, che ha un conflitto tra lo

spostamento di b e la riduzione $B \rightarrow b$:

$B \rightarrow b \bullet B$	$bs \dashv$
$B \rightarrow b \bullet$	$bs \dashv$
$F \rightarrow b \bullet Fe$	e
$B \rightarrow \bullet bB$	$bs \dashv$
$B \rightarrow \bullet b$	$bs \dashv$
$F \rightarrow \varepsilon \bullet$	e
$F \rightarrow \bullet bFe$	e

Ciò mostra la maggiore potenza del metodo $ELR(1)$ rispetto a quello tradizionale, il cui esito dipende dalla particolare grammatica BNF scelta.

Analisi $ELL(1)$ e $ELL(k)$ I m-stati I_3, I_4 contengono più di un candidato nella base, quindi violano la condizione $ELL(1)$. La violazione trova riscontro nelle due derivazioni sinistre

$$\begin{aligned} S &\Rightarrow E \Rightarrow bF \\ S &\Rightarrow E \Rightarrow Fe \Rightarrow bFee \end{aligned}$$

in cui la scelta al secondo passo non è determinata dal carattere corrente, che è sempre b nei due casi.

Prendere $k > 1$ non cambierebbe la situazione perché esistono derivazioni più lunghe delle precedenti in cui sia la mossa di spostamento $0_E \xrightarrow{b} 1_E$, sia la chiamata $0_E \dashrightarrow 0_F$, sono compatibili con la presenza di un numero arbitrario di b .

Soluzione con il metodo tradizionale. Le alternative $E \rightarrow BF \mid FH$ hanno insiemi guida $\{b\}$ e $\{b, e\}$, rispettivamente. Anche aumentando il valore di k gli insiemi guida restano sovrapposti in b^k .

4 Traduzione e analisi semantica 20%

1. Si consideri il linguaggio definito dalla espressione regolare

$$L_1 = ((a|b) d^*)^+$$

Per una frase $x = c_1 d^{m_1} c_2 d^{m_2} \dots c_n d^{m_n} \in L_1$, dove $c_i \in \{a, b\}$ e $m_i \geq 0$ si definisce la traduzione τ :

$$\tau(x) = \tau(c_1 d^{m_1} c_2 d^{m_2} \dots c_n d^{m_n}) = \begin{cases} c_1 c_2 \dots c_n & \text{if } m_1 + m_2 + \dots + m_n \text{ pari} \\ c_n c_{n-1} \dots c_1 & \text{if } m_1 + m_2 + \dots + m_n \text{ dispari} \end{cases}$$

Esempi di traduzioni:

$$x = a d d b d b d d d \xrightarrow{\tau} a b b, \quad y = a d d b b d d d \xrightarrow{\tau} b b a$$

Si risponda alle domande seguenti:

- (a) Si scriva uno schema di traduzione sintattico, ossia una coppia di grammatiche (G_1, G_2) (oppure una grammatica di traduzione) per calcolare la traduzione τ specificata. Si mostrino gli alberi sintattici di y e di $\tau(y)$.
- (b) (facoltativa) Si discuta se sia possibile calcolare la traduzione τ mediante un traduttore a pila *deterministico*.

Soluzione

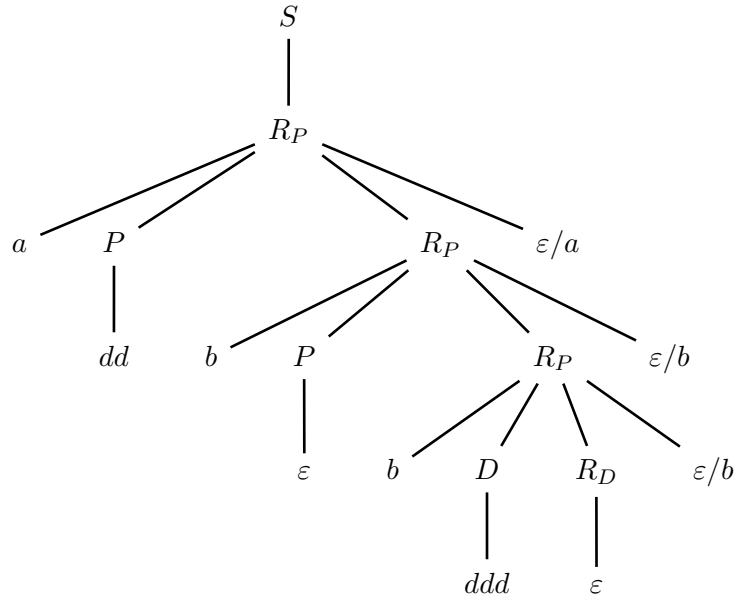
Nel progettare lo schema di traduzione si vede bene che occorre calcolare la classe di parità del numero totale delle lettere d . Poiché la classe di parità sarà nota solo al termine della lettura, mentre va fatta subito la scelta di emettere la stringa abb o la stringa riflessa $(abb)^R$, si capisce che la scelta va fatta alla cieca. In altre parole, il traduttore deve essere nondeterministico: sceglie se emettere abb o $(abb)^R$ e al

termine della lettura verifica se la scelta è corretta in base alla classe di parità. Ecco la grammatica di traduzione G_τ :

$$G_\tau \left\{ \begin{array}{l} S \rightarrow S_P \mid R_P \\ P \rightarrow (dd)^* \\ D \rightarrow d(dd)^* \\ S_P \rightarrow \frac{a}{a}PS_P \mid \frac{b}{b}PS_P \\ S_P \rightarrow \frac{a}{a}DS_D \mid \frac{b}{b}DS_D \mid \varepsilon \\ S_D \rightarrow \frac{a}{a}PS_D \mid \frac{b}{b}PS_D \\ S_D \rightarrow \frac{a}{a}DS_P \mid \frac{b}{b}DS_P \\ R_P \rightarrow aPR_P \frac{\varepsilon}{a} \mid bPR_P \frac{\varepsilon}{b} \\ R_P \rightarrow aDR_D \frac{\varepsilon}{a} \mid bDR_D \frac{\varepsilon}{b} \\ R_D \rightarrow aPR_D \frac{\varepsilon}{a} \mid bPR_D \frac{\varepsilon}{b} \\ R_D \rightarrow aDR_P \frac{\varepsilon}{a} \mid bDR_P \frac{\varepsilon}{b} \mid \varepsilon \end{array} \right.$$

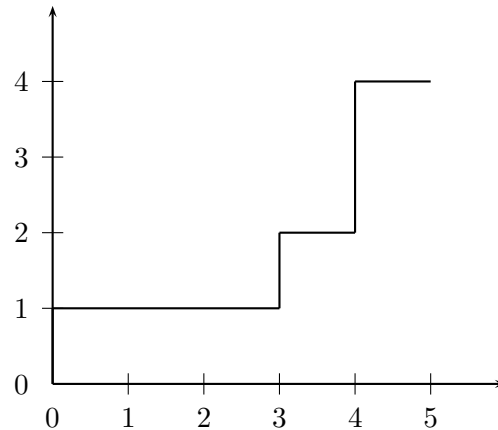
Ovviamente si potrebbe mettere G_τ in forma di schema di traduzione.

L'albero sintattico della stringa $addbbddd$ è:



Lo si potrebbe rappresentare come due alberi sorgente e destinazione separati.

2. Una linea a scalini nel primo quadrante di un piano cartesiano, iniziante dall'origine, è descritta da una sequenza di frasi di tipo $step(x, y)$, dove x e y sono valori interi positivi. Per esempio, la linea in figura



è descritta dalle seguenti istruzioni: $step(3, 1) \ step(1, 1) \ step(1, 2)$. Il linguaggio delle sequenze di frasi è dato dalla seguente grammatica:

$$S \rightarrow L \quad L \rightarrow PL \quad L \rightarrow P \quad P \rightarrow \mathbf{step} \ (x, y)$$

dove x e y sono dei numeri interi positivi.

Si risponda alle domande seguenti:

- Scrivere il più semplice insieme di regole semantiche per una grammatica ad attributi basata su questa sintassi, che calcoli in un attributo $nStep$ della radice il numero di passi di cui è costituita la linea (nell'esempio i passi sono 3).
- Scrivere le regole semantiche per il calcolo, in un attributo della radice, dell'area sottesa dalla linea. Allo scopo si assuma che per i terminali x e y che compaiono in $step(x, y)$ siano disponibili due attributi a valore intero, α e β , rispettivamente corrispondenti al valore numerico delle due costanti.

Consiglio: i suggerisce di far uso dei seguenti attributi:

nome	nonterminali	tipo	significato
yStart	L	dx	ordinata di inizio della linea L
partialArea	L	dx	area parziale sottesa dalla parte di linea che precede L
$\Delta x, \Delta y$	P	sx	spostamento di ascissa e ordinata della frase P (uguali ad α e β)
totalArea	S, L	sx	area totale sottesa dalla linea di L o di S

Si disegni l'albero sintattico decorato con gli attributi per l'esempio precedente.

Soluzione

Grammatica con attributi:

$S \rightarrow L$	$yStart_1 := 0; \quad partialArea_1 := 0;$ $totalArea_0 := totalArea_1;$
$L \rightarrow P L$	$yStart_2 := yStart_0 + \Delta y_1;$ $partialArea_2 := partialArea_0 + yStart_2 \times \Delta x_1;$ $totalArea_0 := totalArea_2;$
$L \rightarrow P$	$totalArea_0 := partialArea_0 + \Delta x_1 * (yStart_0 + \Delta y_1);$
$P \rightarrow \mathbf{step} (x, y)$	$\Delta x_0 := \alpha; \quad \Delta y_0 := \beta;$